

of virtual machines (VMs) and the cloud. The architecture we used also changed. For example, instead of scaling up by adding more CPUs or memory to physical servers, we started 'scaling out' by adding more machines as needed, but, in the cloud. This model gave us the flexibility of an opex-based (operational expenses-based) revenue model. If any of the VMs went down, we got new VMs spawned in minutes. In short, we started treating servers as 'cattle' and not 'pets'.

However, the cloud and virtualisation came with their own problems and still have many limitations. We are still spending a lot of time managing them — for example, bringing VMs up and down, based on need. We have to architect for availability and fault-tolerance, size workloads, and manage capacity and utilisation. If we have dedicated VMs provisioned in the cloud, we still have to pay for the reserved resources (even if it's just idle time). Hence, moving from a capex model to an opex one is not enough. What we need is to only pay for what we are using (and not more than that) and 'pay as you go'. Serverless computing promises to address exactly this problem.

The other key aspect is agility. Businesses today need to be very agile. Technology complexity and infrastructure operations cannot be used as an excuse for not delivering value at scale. Ideally, much of the engineering effort should be focused on providing functionality that delivers the desired experience, and not in monitoring and managing the infrastructure that supports the scale requirements. This is where serverless shines.

What is serverless?

Consider a chatbot for booking movie tickets - let's call it MovieBot. Any user can make queries about movies, book tickets, or cancel them in a conversational style (e.g., "Is 'Dunkirk' playing in Urvasi Theatre in Bengaluru tonight?" in voice or text).

This solution requires three elements: a chat interface channel (like Skype or Facebook Messenger), a natural language processor (NLP) to understand the user's intentions (e.g., 'book a ticket', 'ticket availability', 'cancellation', etc), and then access to a back-end where the transactions and data pertaining to movies is stored. The chat interface channels are universal and can be used for different kinds of bots. NLP can be implemented using technologies like AWS Lex or IBM Watson. The question is: how is the back-end served? Would you set up a dedicated server (or a cluster of servers), an API gateway, deploy load balancers, or put in place identity and access control mechanisms? That's costly and painful, right! That's where serverless technology can help.

The solution is to set up some compute capacity to process data from a database and also execute this logic in a language of choice. For example, if you are using the AWS platform, you can use DynamoDB for the back-end, write programming

logic as Lambda functions, and expose them through the AWS API Gateway with a load balancer. This entire set-up does not require you to provision any infrastructure or have any knowledge about underlying servers/VMs in the cloud. You can use a database of your choice for the back-end. Then choose any programming language supported in AWS Lambda, including Java, Python, JavaScript, and C#. There is no cost involved if there aren't any

users using the MovieBot. If a blockbuster like 'Baahubali' is released, then there could be a huge surge in users accessing the MovieBot at the same time, and the set-up would effortlessly scale (you have to pay for the calls, though). Phew! You essentially engineered a serverless application.

With this, it's time to define the term 'serverless'.

Serverless architectures refer to applications that significantly depend on third-party services (known as Backend-as-a-Service or BaaS) or on custom code that's run in ephemeral containers (Function-as-a-Service or FaaS).

Hmm, that's a mouthful of words; so let's dissect this description.

- **Backend-as-a-Service:** Typically, databases (often NoSQL flavours) hold the data and can be accessed over the cloud, and a service can be used to help access that back-end. Such a back-end service is referred to as BaaS.
- **Function-as-a-Service:** Code that processes the requests (i.e., the 'programming logic' written in your favourite programming language) could be run on containers that are spun and destroyed as needed. They are known as FaaS.

The word 'serverless' is misleading because it literally means there are no servers. Actually, the word implies, "I don't care what a server is." In other words, serverless enables us to create applications without thinking about servers, i.e., we can build and run applications or services without worrying about provisioning, managing or scaling the underlying infrastructure. Just put your code in the cloud and run it! Keep in mind that this applies to Platform-as-a-Service (PaaS) as well; although you may not deal with direct VMs with PaaS, you still have to deal with instance sizes and capacity.

Think of serverless as a piece of functionality to run — not in your machine but executed remotely. Typically, serverless functions are executed in an 'event-driven' fashion — the functions get executed as a response to events or requests on HTTP. In the case of the MovieBot, the Lambda functions are invoked to serve user queries as and when user(s) interact with it.

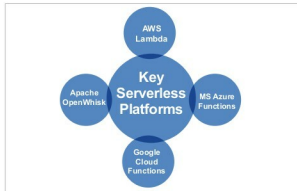


Figure 1: Key serverless platforms